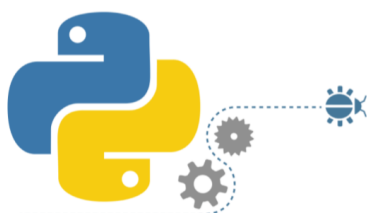


Python基础知识

1.认识Python

1.1 Python概述

1.1.1 基本概念



Python is a programming language that lets you work quickly and integrate systems more effectively.

- Python是一门解释型、面向对象的高级编程语言.
- Python是开源免费的、支持交互式、可跨平台移植的脚本语言.

★ 诞生和发展

- 1991年，第一个Python编译器(同时也是解释器)诞生。它是用C语言实现的，并能够调用C库(.so文件)。从一出生，Python已经具有了：类，函数，异常处理，包含表和词典在内的核心数据类型，以及模块为基础的拓展系统。
- 2000年，Python 2.0 由 BeOpen PythonLabs 团队发布，加入内存回收机制，奠定了Python语言框架的基础
- 2008年，Python 3 在一个意想不到的情况下发布了，对语言进行了彻底的修改，没有向后兼容

1.1.2 语言优势

2020年2月的TIOBE的编程语言排行榜：

Python的设计混合了传统语言的软件工程的特点和脚本语言的易用性，具有如下**特性**：

- 开源、易于维护
- 可移植
- 易于使用、简单优雅
- 广泛的标准库、功能强大
- 可扩展、可嵌入
- ...



Python也存在缺点：

◆ 运行速度慢

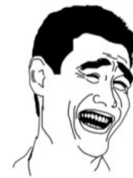
- Python是**解释型语言**，运行时翻译为机器码非常耗时，而C语言是运行前直接编译成CPU能执行的机器码。但是大量的应用程序不需要这么快的运行速度，因为**用户根本感觉不出来**。



不要在意程序运行速度

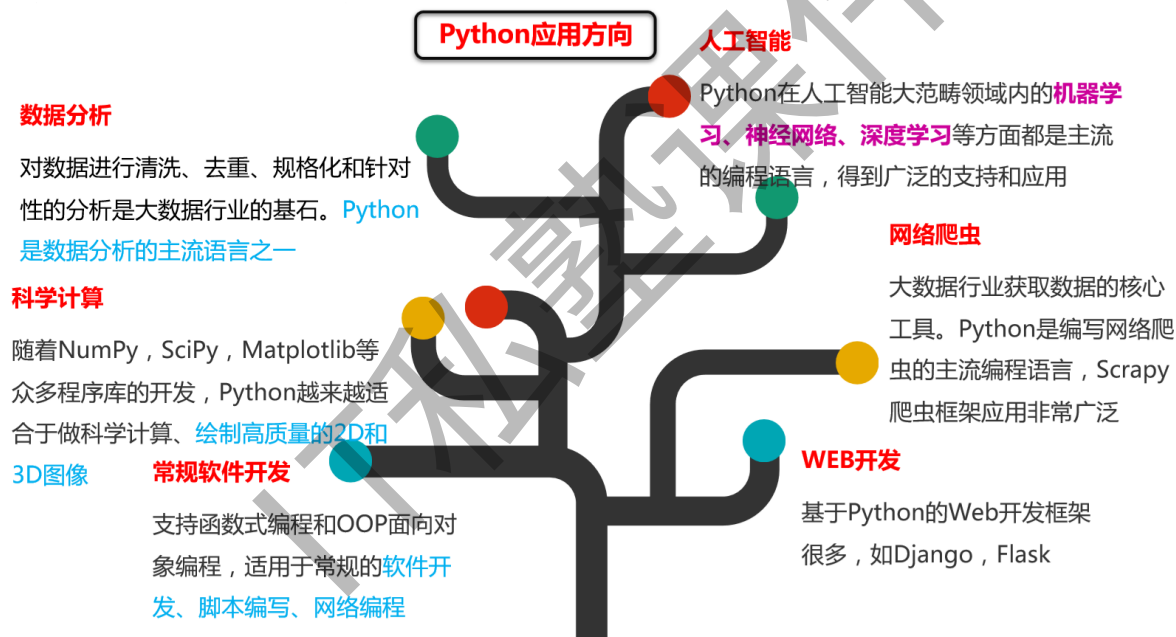
◆ 代码不能加密

- 解释型语言发布程序就是发布源代码，而C语言只需要把编译后的机器码发布出去，从机器码反推出C代码是不可能的



大家都那么忙，
哪有闲功夫破解你的烂代码

1.1.3 典型应用



2020-02-25_20-19-142

1.2 安装Python环境

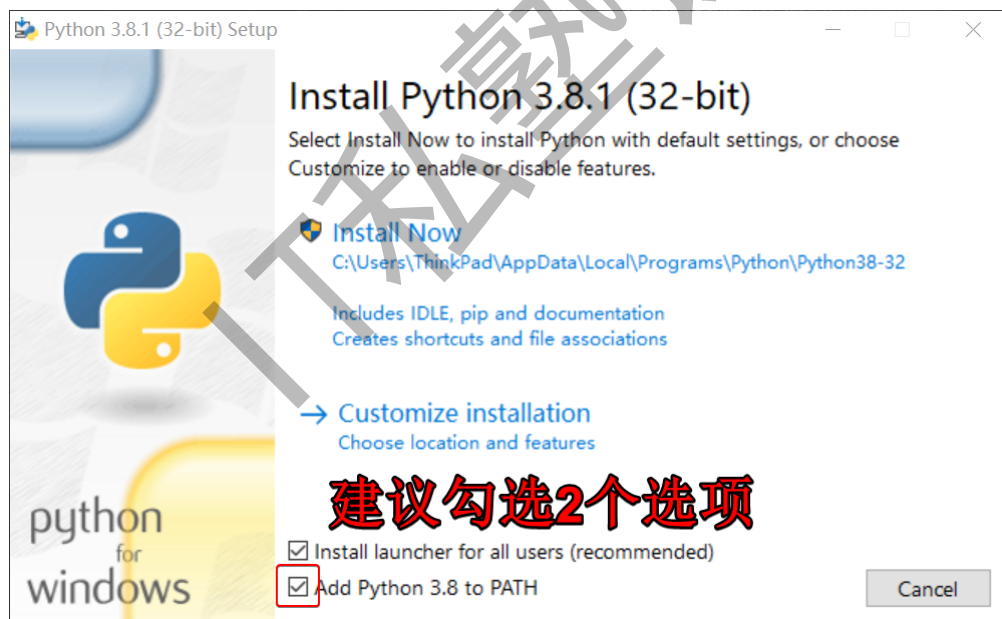
- ◆ Python是跨平台的，可以运行在Windows、Mac和各种Unix/Linux系统上。
- ◆ Python有两个版本，一个是2.x版，一个是3.x版，这两个版本是**不兼容的**，本教程以Python3.5版本为基础（Windows上安装时注意添加环境变量）。
- ◆ Python代码是以.py为扩展名的文本文件，要运行代码，需要安装Python解释器：
- ◆ CPython
官方默认编译器，安装Python后直接获得该解释器，以>>>作为提示符
- ◆ IPython
基于CPython的一个交互式解释器，用In [序号]: 作为提示符

1.2.1 下载Python

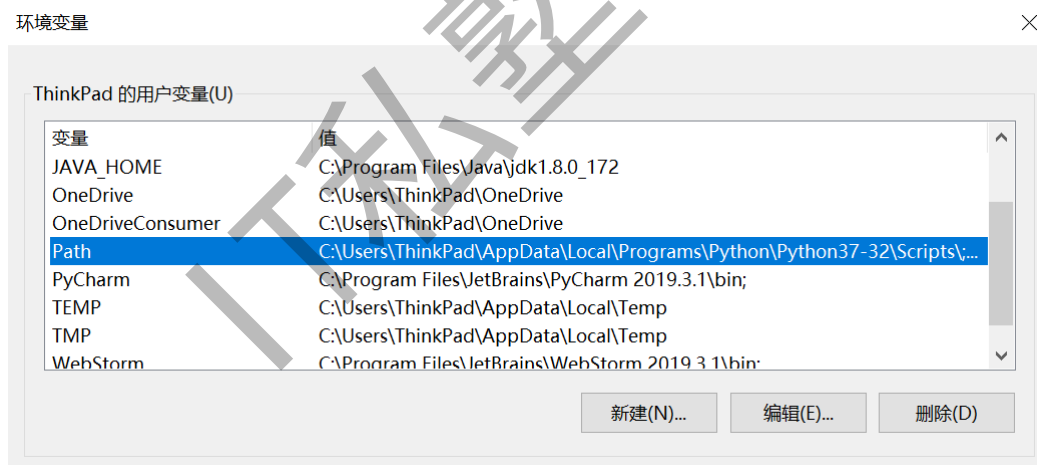
<https://www.python.org/downloads/>

目前最新版是3.8.2

1.2.2 安装Python



1.2.3 配置环境变量

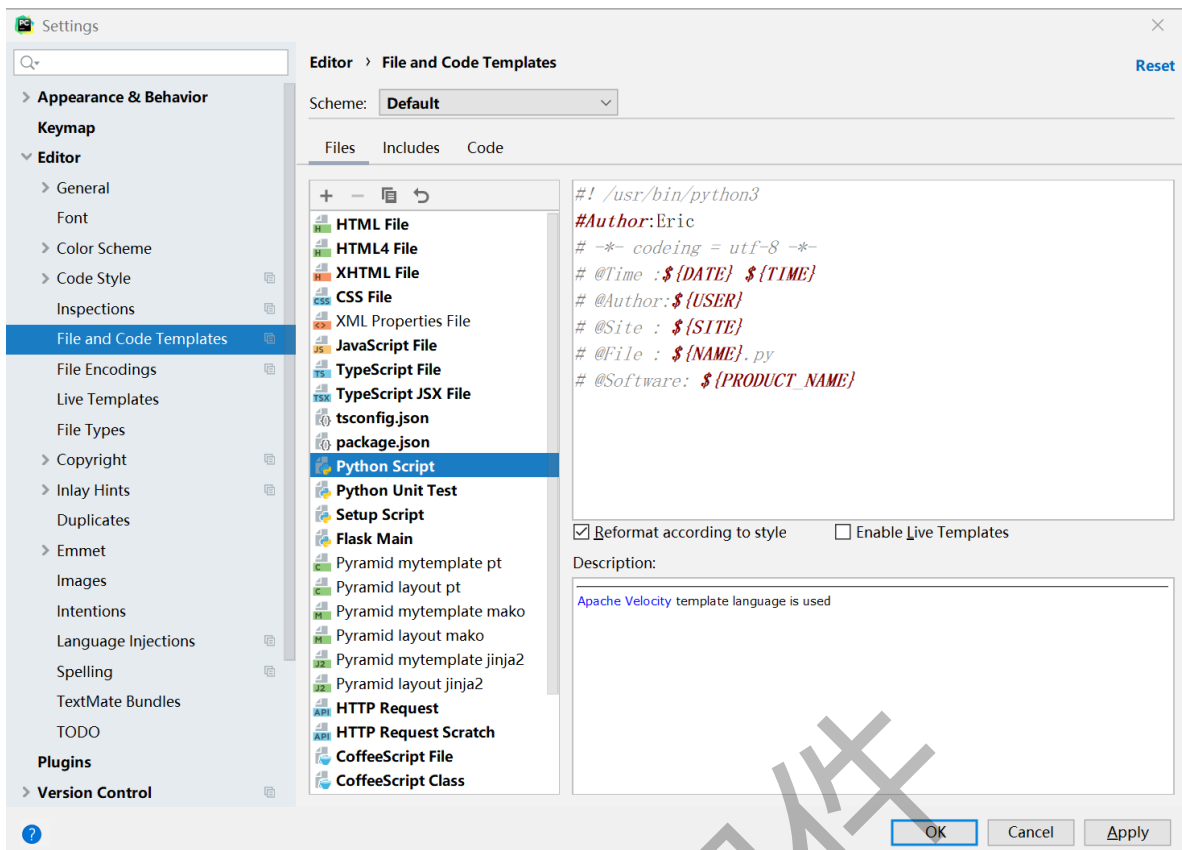


Python的安装路径，及其Script文件夹

1.2.4 下载并配置Pycharm

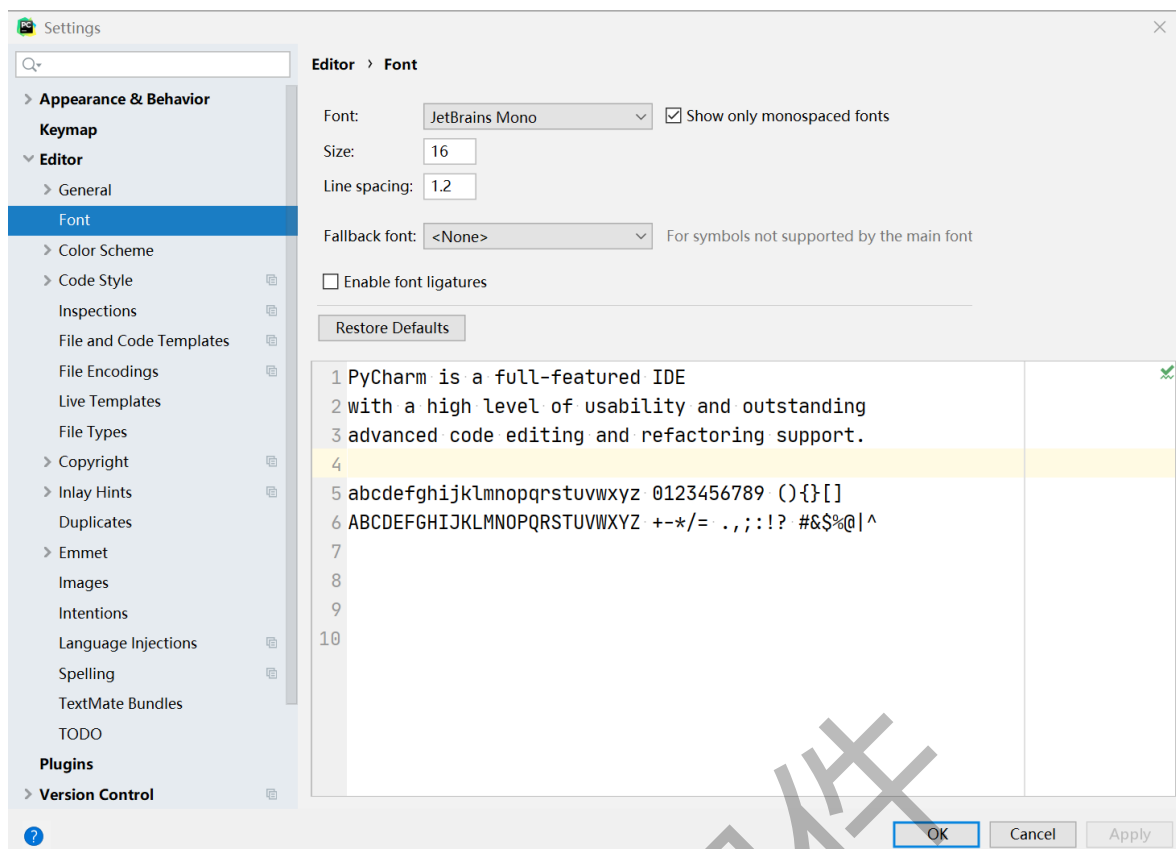
<https://www.jetbrains.com/pycharm/download/#section=windows>

- Python文件模板配置



```
# -*- coding = utf-8 -*-
# @Time :${DATE} ${TIME}
# @Author:Eric
# @File : ${NAME}.py
# @Software: ${PRODUCT_NAME}
```

- 字体设置



课堂练习：

下载并安装Python环境（10分钟）

1.3 编写第一个Python程序

- ◆ **交互模式**：在命令行敲击命令 `python`，即可进入Python交互模式，提示符是`>>>`。
- ◆ **命令模式**：在Python交互模式下输入 `exit()`，就退出了Python交互模式，回到命令行模式

```
C:\> python
Python 3.7.3 ...
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

```
>>> print('hello world')
hello world
```

```
C:\> python hello.py
```

1.4 注释

- 单行注释

以#开头，#右边的所有东西当做说明，而不是真正要执行的程序，起辅助说明作用

```
# 我是注释，可以在里写一些功能说明之类的哦  
print('hello world')
```

- 多行注释

```
'''  
    我是多行注释，  
    可以写很多很多行的功能说明  
  
    哈哈哈哈哈。。。  
'''  
print('hello world')
```

- python程序中，中文支持

如果直接在程序中用到了中文，比如

```
print('你好')
```

如果直接运行输出，程序会出错。

解决的办法为：在程序的开头写入如下代码，这就是中文注释

```
#coding=utf-8
```

修改之后的程序：

```
#coding=utf-8  
print('你好')
```

运行结果：

```
你好
```

注意：

在python的语法规范中推荐使用的方式：

```
# -*- coding:utf-8 -*-
```

课堂练习：

编写第一个Python程序，并配置好Pycharm（10分钟）

1.5 变量及类型

- 变量可以是任意的数据类型，在程序中用一个变量名表示
- 变量名必须是大小写英文、数字和下划线（_）的组合，且不能以数字开头，如：

```
>>> a = 1 # 变量a是一个整数
>>> t_007 = 'T007' # 变量t_007是一个字符串
```

- 赋值（比如a= 'ABC'）时，Python解释器干了两件事
 - 在内存中创建一个'ABC'的字符串
 - 在内存中创建一个名为a的变量，并把它指向'ABC'



1.6 标识符和关键字

- 什么是关键字

python一些具有特殊功能的标示符，这就是所谓的关键字

关键字，是python已经使用的了，所以不允许开发者自己定义和关键字相同的名字的标示符

- 查看关键字:

```
>>> import keyword
>>> keyword.kwlist
```

and	as	assert	break	class	continue	def
del	elif	else	except	exec	finally	for
from	global	if	in	import	is	lambda
not	or	pass	print	raise	return	try
while	with	yield				

1.7 输出

1.7.1 普通输出

- python中普通的输出

```
# 打印提示
print('hello world')
```

1.7.2 格式化输出

1.7.2.1 格式化操作的目的

比如有以下代码:

```
print("我今年10岁")
print("我今年11岁")
print("我今年12岁")
...
```

- 想一想:

在输出年龄的时候,用了多次"我今年xx岁",能否简化一下程序呢???

- 答:

字符串格式化

1.7.2.2 什么是格式化

看如下代码:

```
age = 10
print("我今年%d岁"%age)

age += 1
print("我今年%d岁"%age)

age += 1
print("我今年%d岁"%age)

...
```

在程序中,看到了%这样的操作符,这就是Python中格式化输出。

```
age = 18
name = "xiaohua"
print("我的姓名是%s,年龄是%d"%(name,age))
```

1.7.2.3 常用的格式符号

下面是完整的,它可以与%符号使用列表:

格式符号	转换
%c	字符
%s	通过str() 字符串转换来格式化
%i	有符号十进制整数
%d	有符号十进制整数
%u	无符号十进制整数
%o	八进制整数
%x	十六进制整数（小写字母）
%X	十六进制整数（大写字母）
%e	索引符号（小写'e'）
%E	索引符号（大写'E'）
%f	浮点实数
%g	%f和%e 的简写
%G	%f和%E的简写

1.7.3 换行输出

在输出的时候，如果有 `\n` 那么，此时 `\n` 后的内容会在另外一行显示

```
print("1234567890-----") # 会在一行显示

print("1234567890\n-----") # 一行显示1234567890，另外一行显示-----
```

1.8 输入

```
password = input("请输入密码:")
print('您刚刚输入的密码是:', password)
```

运行结果：

- `input()`的小括号中放入的是，提示信息，用来在获取数据之前给用户的一个简单提示
- `input()`在从键盘获取了数据以后，会存放到等号左边的变量中

- input()函数接受的输入必须是表达式

```
>>> a = input()
123
>>> a
123
>>> type(a)
<type 'int'>
>>> a = input()
abc
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<string>", line 1, in <module>
NameError: name 'abc' is not defined
>>> a = input()
"abc"
>>> a
'abc'
>>> type(a)
<type 'str'>
>>> a = input()
1+3
>>> a
4
>>> a = input()
"abc"+"def"
>>> a
'abcdef'
>>> value = 100
>>> a = input()
value
>>> a
100
```

1.9运算符和表达式

算术运算符

以下假设变量a为10，变量b为21：

运算符	描述	实例
+	加 - 两个对象相加	a + b 输出结果 31
-	减 - 得到负数或是一个数减去另一个数	a - b 输出结果 -11
*	乘 - 两个数相乘或是返回一个被重复若干次的字符串	a * b 输出结果 210
/	除 - x 除以 y	b / a 输出结果 2.1
%	取模 - 返回除法的余数	b % a 输出结果 1
**	幂 - 返回x的y次幂	a ** b 为10的21次方
//	取整除 - 向下取接近除数的整数	9 // 2 的结果是 4 -9 // 2 的结果是 -5

比较运算符

以下假设变量a为10，变量b为20：

运算符	描述	实例
==	等于 - 比较对象是否相等	(a == b) 返回 False
!=	不等于 - 比较两个对象是否不相等	(a != b) 返回 True
>	大于 - 返回x是否大于y	(a > b) 返回 False
<	小于 - 返回x是否小于y。所有比较运算符返回1表示真，返回0表示假。这分别与特殊的变量True和False等价	(a < b) 返回 True
>=	大于等于 - 返回x是否大于等于y	(a >= b) 返回 False
<=	小于等于 - 返回x是否小于等于y	(a <= b) 返回 True

赋值运算符

以下假设变量a为10，变量b为20：

运算符	描述	实例
=	简单的赋值运算符	c = a + b 将 a + b 的运算结果赋值为 c
+=	加法赋值运算符	c += a 等效于 c = c + a
-=	减法赋值运算符	c -= a 等效于 c = c - a
*=	乘法赋值运算符	c *= a 等效于 c = c * a
/=	除法赋值运算符	c /= a 等效于 c = c / a
%=	取模赋值运算符	c %= a 等效于 c = c % a
**=	幂赋值运算符	c **= a 等效于 c = c ** a
//=	取整除赋值运算符	c //= a 等效于 c = c // a

(了解)

位运算符

运算符	描述	实例
&	按位与运算符：参与运算的两个值,如果两个相应位都为1,则该位的结果为1,否则为0	(a & b) 输出结果 12 ，二进制解释：0000 1100
	按位或运算符：只要对应的二个二进位有一个为1时，结果位就为1。	(a b) 输出结果 61 ，二进制解释：0011 1101
^	按位异或运算符：当两对应的二进位相异时，结果为1	(a ^ b) 输出结果 49 ，二进制解释：0011 0001
~	按位取反运算符：对数据的每个二进位取反,即将1变为0,把0变为1。~x类似于 -x-1	(~a) 输出结果 -61 ，二进制解释：1100 0011， 在一个有符号二进制的补码形式。
<<	左移动运算符：运算数的各二进位全部左移若干位，由"<<"右边的数指定移动的位数，高位丢弃，低位补0。	a << 2 输出结果 240 ，二进制解释：1111 0000
>>	右移动运算符：把">>"左边的运算数的各二进位全部右移若干位，">>"右边的数指定移动的位数	a >> 2 输出结果 15 ，二进制解释：0000 1111

(了解)

逻辑运算符

以下假设变量a为10，变量b为20：

运算符	描述	实例
and	布尔“与” - 如果 x 为 False，x and y 返回 False，否则它返回 y 的计算值	(a and b) 返回 20
or	布尔“或” - 如果 x 是 True，它返回 x 的值，否则它返回 y 的计算值	(a or b) 返回 10
not	布尔“非” - 如果 x 为 True，返回 False。如果 x 为 False，它返回 True	not(a and b) 返回 False

成员运算符

运算符	描述	实例
in	如果在指定的序列中找到值返回 True，否则返回 False	x 在 y 序列中，如果 x 在 y 序列中返回 True
not in	如果在指定的序列中没有找到值返回 True，否则返回 False	x 不在 y 序列中，如果 x 不在 y 序列中返回 True

(了解)

身份运算符

运算符	描述	实例
is	is 是判断两个标识符是不是引用自一个对象	x is y , 类似 id(x) == id(y) , 如果引用的是同一个对象则返回 True, 否则返回 False
is not	is not 是判断两个标识符是不是引用自不同对象	x is not y , 类似 id(a) != id(b) 。如果引用的不是同一个对象则返回结果 True, 否则返回 False。

注意: **id(x)** 函数用于获取对象内存地址

运算符优先级

```
>>> a = 20
>>> b = 10
>>> c = 15
>>> d = 5
>>> e = (a + b) * c / d
>>> print(e)
90.0
>>> e = (a + b) * (c / d)
>>> print(e)
90.0
>>> e = a + (b * c) / d
>>> print(e)
50.0
```

运算符	描述
**	指数 (最高优先级)
*/%//	乘, 除, 取模和取整除
+ -	加法减法
>><<	右移, 左移运算符
&	位 'AND'
^ 	位运算符
<= < > >=	比较运算符
<> == !=	等于运算符
= %= /= //= -= += *= **=	赋值运算符
is is not	身份运算符
in not in	成员运算符
and or not	逻辑运算符

2.判断语句和循环语句

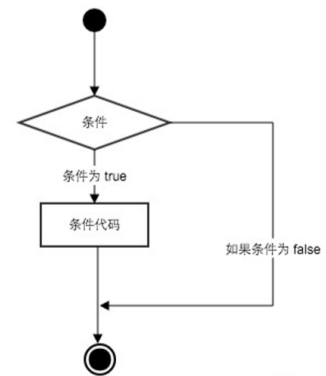
2.1 条件判断语句

🔗 条件判断

- 计算机之所以能做很多自动化的任务，因为它可以自己做**条件判断**。
- Python指定任何**非0和非空值为True**，**0 或者 None为False**
- Python 编程中 **if 语句**用于控制程序的执行，基本形式为：

```
if 判断条件1:  
    执行语句1  
elif 判断条件2:  
    执行语句2  
else:  
    执行语句3
```

- ✓ 其中"判断条件"成立时（非零），则执行后面的语句，而执行内容可以多行，以**缩进**来区分表示同一范围。
- ✓ else 为可选语句，当需要在条件不成立时执行内容则可以执行相关语句。



- if语句是用来进行判断的，其使用格式如下：

if 要判断的条件：
条件成立时，要做的事情

- demo:

```
age = 30  
  
print "-----if判断开始-----"  
  
if age>=18:  
    print "我已经成年了"  
  
print "-----if判断结束-----"
```

- 运行结果:

```
-----if判断开始-----  
我已经成年了  
-----if判断结束-----
```

2.1.1 if-else

- 代码的缩进为一个tab键，或者4个空格

```
if True:  
    print ("Answer")  
    print ("True")  
else:  
    print ("Answer")  
    print ("False")    # 缩进不一致，会导致运行错误
```

2.1.2 elif

- demo:

```
score = 77

if score >= 90 and score <= 100:
    print('本次考试, 等级为A')
elif score >= 80 and score < 90:
    print('本次考试, 等级为B')
elif score >= 70 and score < 80:
    print('本次考试, 等级为C')
elif score >= 60 and score < 70:
    print('本次考试, 等级为D')
# elif score >= 0 and score < 60:
else:
    print('本次考试, 等级为E') # elif可以else一起使用
```

注意

- 可以和else一起使用
- elif必须和if一起使用, 否则出错

2.1.3 if嵌套

2.1.3.1 if嵌套的格式

```
if 条件1:
    满足条件1 做的事情1
    满足条件1 做的事情2
    ... (省略) ...

    if 条件2:
        满足条件2 做的事情1
        满足条件2 做的事情2
        ... (省略) ...
```

- 说明
 - 外层的if判断, 也可以是if-else
 - 内层的if判断, 也可以是if-else
 - 根据实际开发的情况, 进行选择

2.1.3.2 if嵌套的应用

demo:

```

xingBie = 1      # 用1代表男生，0代表女生
danShen = 1      # 用1代表单身，0代表有男/女朋友

if xingBie == 1:
    print("是男生")
    if danShen == 1:
        print("我给你介绍一个吧? ")
    else:
        print("你给我介绍一个呗? ")
else:
    print("你是女生")
    print(".....")

```

2.1.3.2 import 与 from...import

在 python 用 `import` 或者 `from...import` 来导入相应的模块。

将整个模块(somemodule)导入, 格式为: `import somemodule`

从某个模块中导入某个函数,格式为: `from somemodule import somefunction`

从某个模块中导入多个函数,格式为: `from somemodule import firstfunc, secondfunc, thirdfunc`

将某个模块中的全部函数导入, 格式为: `from somemodule import *`

【生成随机数】

1.第一行代码引入库:

```
import random      #引入随机库
```

2.生成指定范围的随机数

```
computer = random.randint(0,2)  #随机生成0、1、2中的一个数字，赋值给变量computer
```

课堂练习:

综合使用if语句的相关知识, 实现石头剪子布游戏效果。显示下面提示信息:

请输入: 剪刀 (0)、石头 (1)、布 (2): _

用户输入数字0-2中的一个数字, 与系统随机生成的数字比较后给出结果信息。

例如: 输入0后, 显示如下

你的输入为：剪刀（0）
随机生成数字为：1
哈哈，你输了：)

提示：对于输入不正常的情况尽可能考虑全面，使程序能够正常运行。

建议用时15~20分钟。

2.2 循环语句

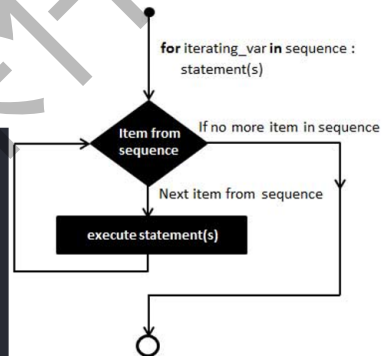
2.2.1 for循环

🔗 循环

Python的循环有两种，一种是for...in循环，可以依次把list或tuple中的元素迭代出来

```
>>> for i in range(5):  
...     print(i)  
...  
0  
1  
2  
3  
4
```

```
>>> for i in range(0, 10, 3):  
...     print(i)  
...  
0  
3  
6  
9
```



for循环的格式

for 临时变量 **in** 列表或者字符串等：
循环满足条件时执行的代码

demo:

```
name = 'chengdu'  
  
for x in name:  
    print(x)
```

2.2.2 while循环

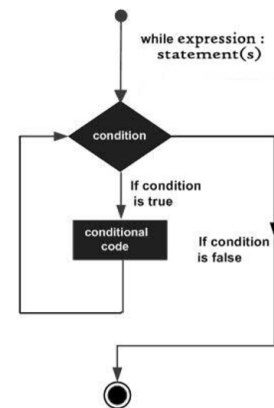
循环

第二种是while，只要条件满足，就不断循环，条件不满足时退出循环

在 while ... else 在条件语句为 false 时执行 else 的语句块

```
>>> sum = 0
>>> n = 99
>>> while n > 0:
...     sum = sum + n
...     n = n - 2
...
>>> print(sum)
2500

>>> count = 0
>>> while count < 3:
...     print(count, "小于 3")
...     count = count + 1
... else:
...     print(count, "大于或等于 3")
...
0 小于 3
1 小于 3
2 小于 3
3 大于或等于 3
```



demo

```
i = 0
while i < 5:
    print("当前是第%d次执行循环"%(i+1))
    print("i=%d"%i)
    i+=1
```

课堂练习：

1. 计算1~100的累积和（包含1和100）
 2. 计算1~100之间偶数的累积和（包含1和100）
- 建议用时20分钟。

2.2.3 break和continue

🔗 break、continue、pass语句

- break语句可以跳出 for 和 while 的循环体
- continue语句跳过当前循环，直接进行下一轮循环
- pass是空语句，一般用做占位语句，不做任何事情

```
>>> n = 1
>>> while n <= 100:
...     if n > 10:
...         break
...     print(n)
...     n += 1
```

结果：打印1~10

```
>>> n = 1
>>> while n < 10:
...     n = n + 1
...     if n % 2 == 0:
...         continue
...     print(n)
```

结果：打印1,3,5,7,9

```
>>> for letter in 'Room':
...     if letter == 'o':
...         pass
...     print('pass')
...     print(letter)
...
... R
pass
o
pass
o
m
```

- 带有break的循环示例如下：

```
i = 0

while i < 10:
    i = i + 1
    print('----')
    if i == 5:
        break
    print(i)
```

- 带有continue的循环示例如下：

```
i = 0

while i < 10:
    i = i + 1
    print('----')
    if i == 5:
        continue
    print(i)
```

作业：

使用for循环和while循环，打印九九乘法表，显示效果如下图：

```
1*1=1
2*1=2 2*2=4
3*1=3 3*2=6 3*3=9
4*1=4 4*2=8 4*3=12 4*4=16
5*1=5 5*2=10 5*3=15 5*4=20 5*5=25
6*1=6 6*2=12 6*3=18 6*4=24 6*5=30 6*6=36
7*1=7 7*2=14 7*3=21 7*4=28 7*5=35 7*6=42 7*7=49
8*1=8 8*2=16 8*3=24 8*4=32 8*5=40 8*6=48 8*7=56 8*8=64
9*1=9 9*2=18 9*3=27 9*4=36 9*5=45 9*6=54 9*7=63 9*8=72 9*9=81
```

3.字符串、列表、元组、字典

3.1 字符串

🧩 Python的核心数据类型

◆ String (字符串)

- Python中的字符串可以使用单引号、双引号和三引号（三个单引号或三个双引号）括起来，使用反斜杠 \ 转义特殊字符
- Python3源码文件默认以UTF-8编码，所有字符串都是unicode字符串
- 支持字符串拼接、截取等多种运算

```
>>> a = "Hello"
>>> b = "Python"
>>> print("a + b 输出结果 :", a + b)
a + b 输出结果 : HelloPython
>>> print("a[1:4] 输出结果 :", a[1:4])
a[1:4] 输出结果 : ell
```

```
word = '字符串'
sentence = "这是一个句子。"
paragraph = """这是一个段落，
可以由多行组成"""
```

3.1.1 单引号和双引号如何选择？

- 1、包含单引号的字符串

假如你想定义一个字符串my_str，其值为： I'm a student，则可以采用如下方式，通过转义字符\进行定义。

```
my_str = 'I\'m a student'
```

也可以不使用转义字符，利用双引号直接进行定义。

```
my_str = "I'm a student"
```

2、包含双引号的字符串

假如你想定义一个字符串my_str，其值为： Jason said "I like you"，则可以采用如下方式，通过转义字符\进行定义。

```
my_str = "Jason said \"I like you\""
```

也可以不使用转义字符，利用单引号直接进行定义。

```
my_str = 'Jason said "I like you"'
```

3.1.2 转义字符

转义字符	描述
(在行尾时)	续行符
\\	反斜杠符号
\'	单引号
\"	双引号
\a	响铃
\b	退格(Backspace)
\000	空
\n	换行
\v	纵向制表符
\t	横向制表符
\r	回车
\f	换页
\oyy	八进制数，yy 代表的字符，例如：\o12 代表换行，其中 o 是字母，不是数字 0。
\xyy	十六进制数，yy代表的字符，例如：\x0a代表换行
\other	其它的字符以普通格式输出

3.1.3 字符串的截取和连接

```
str='chengdu'

print(str)                # 输出字符串
print(str[0:-1])          # 输出第一个到倒数第二个的所有字符
print(str[0])              # 输出字符串第一个字符
print(str[2:5])            # 输出从第三个开始到第五个的字符
print(str[2:])             # 输出从第三个开始后的所有字符
print(str * 2)             # 输出字符串两次
print(str + '你好')        # 连接字符串
print(str[:5])             # 输出第五个字母前的所有字符
print(str[0:7:2])          # [起始：终止：步长]

print('-----')

print('hello\nchengdu')    # 使用反斜杠(\)+n转义特殊字符
print(r'hello\npython')    # 在字符串前面添加一个 r，表示原始字符串，不会发生转义
```

3.1.4 字符串的常见操作

序号	方法及描述
1	capitalize() 将字符串的第一个字符转换为大写
2	center(width, fillchar) 返回一个指定的宽度 width 居中的字符串，fillchar 为填充的字符，默认为空格。
3	count(str, beg= 0,end=len(string)) 返回 str 在 string 里面出现的次数，如果 beg 或者 end 指定则返回指定范围内 str 出现的次数
4	bytes.decode(encoding="utf-8", errors="strict") Python3 中没有 decode 方法，但我们可以使用 bytes 对象的 decode() 方法来解码给定的 bytes 对象，这个 bytes 对象可以由 str.encode() 来编码返回。
5	encode(encoding='UTF-8',errors='strict') 以 encoding 指定的编码格式编码字符串，如果出错默认报一个ValueError 的异常，除非 errors 指定的是'ignore'或者'replace'
6	endswith(suffix, beg=0, end=len(string)) 检查字符串是否以 obj 结束，如果beg 或者 end 指定则检查指定的范围内是否以 obj 结束，如果是，返回 True,否则返回 False。
7	expandtabs(tabsize=8) 把字符串 string 中的 tab 符号转为空格，tab 符号默认的空格数是 8。

序号	方法及描述
8	<code>find(str, beg=0, end=len(string))</code> 检测 str 是否包含在字符串中, 如果指定范围 beg 和 end, 则检查是否包含在指定范围内, 如果包含返回开始的索引值, 否则返回-1
9	<code>index(str, beg=0, end=len(string))</code> 跟find()方法一样, 只不过如果str不在字符串中会报一个异常.
10	isalnum() 如果字符串至少有一个字符并且所有字符都是字母或数字则返回 True,否则返回 False
11	isalpha() 如果字符串至少有一个字符并且所有字符都是字母则返回 True, 否则返回 False
12	isdigit() 如果字符串只包含数字则返回 True 否则返回 False..
13	<code>islower()</code> 如果字符串中包含至少一个区分大小写的字符, 并且所有这些(区分大小写的)字符都是小写, 则返回 True, 否则返回 False
14	isnumeric() 如果字符串中只包含数字字符, 则返回 True, 否则返回 False
15	<code>isspace()</code> 如果字符串中只包含空白, 则返回 True, 否则返回 False.
16	<code>istitle()</code> 如果字符串是标题化的(见 title())则返回 True, 否则返回 False
17	<code>isupper()</code> 如果字符串中包含至少一个区分大小写的字符, 并且所有这些(区分大小写的)字符都是大写, 则返回 True, 否则返回 False
18	join(seq) 以指定字符串作为分隔符, 将 seq 中所有的元素(的字符串表示)合并为一个新的字符串
19	len(string) 返回字符串长度
20	<code>[ljust(width, fillchar)]</code> 返回一个原字符串左对齐,并使用 fillchar 填充至长度 width 的新字符串, fillchar 默认为空格.
21	<code>lower()</code> 转换字符串中所有大写字符为小写.
22	lstrip() 截掉字符串左边的空格或指定字符.
23	<code>maketrans()</code> 创建字符映射的转换表, 对于接受两个参数的最简单的调用方式, 第一个参数是字符串, 表示需要转换的字符, 第二个参数也是字符串表示转换的目标.
24	<code>max(str)</code> 返回字符串 str 中最大的字母.
25	<code>min(str)</code> 返回字符串 str 中最小的字母.
26	<code>[replace(old, new [,max])]</code> 把 将字符串中的 str1 替换成 str2,如果 max 指定, 则替换不超过 max 次.
27	<code>rfind(str, beg=0,end=len(string))</code> 类似于 find()函数, 不过是从右边开始查找.
28	<code>rindex(str, beg=0, end=len(string))</code> 类似于 index(), 不过是从右边开始.
29	<code>[rjust(width,, fillchar)]</code> 返回一个原字符串右对齐,并使用fillchar(默认空格) 填充至长度 width 的新字符串
30	rstrip() 删除字符串字符串末尾的空格.

序号	方法及描述
31	<code>split(str="", num=string.count(str))</code> 以 <code>str</code> 为分隔符截取字符串, 如果 <code>num</code> 有指定值, 则仅截取 <code>num+1</code> 个子字符串
32	<code>[splitlines(keepends)]</code> 按照行(<code>'\r'</code> , <code>'\r\n'</code> , <code>'\n'</code>)分隔, 返回一个包含各行作为元素的列表, 如果参数 <code>keepends</code> 为 <code>False</code> , 不包含换行符, 如果为 <code>True</code> , 则保留换行符。
33	<code>startswith(substr, beg=0,end=len(string))</code> 检查字符串是否是以指定子字符串 <code>substr</code> 开头, 是则返回 <code>True</code> , 否则返回 <code>False</code> 。如果 <code>beg</code> 和 <code>end</code> 指定值, 则在指定范围内检查。
34	<code>[strip(chars)]</code> 在字符串上执行 <code>lstrip()</code> 和 <code>rstrip()</code>
35	<code>swapcase()</code> 将字符串中大写转换为小写, 小写转换为大写
36	<code>title()</code> 返回"标题化"的字符串,就是说所有单词都是以大写开始, 其余字母均为小写(见 <code>istitle()</code>)
37	<code>translate(table, deletechars="")</code> 根据 <code>str</code> 给出的表(包含 256 个字符)转换 <code>string</code> 的字符, 要过滤掉的字符放到 <code>deletechars</code> 参数中
38	<code>upper()</code> 转换字符串中的小写字母为大写
39	<code>zfill (width)</code> 返回长度为 <code>width</code> 的字符串, 原字符串右对齐, 前面填充0
40	<code>isdecimal()</code> 检查字符串是否只包含十进制字符, 如果是返回 <code>true</code> , 否则返回 <code>false</code> 。

3.2 列表

Python的核心数据类型

◆ List (列表)

- 列表可以完成大多数集合类的数据结构实现。列表中元素的类型可以不相同, 它支持数字, 字符串甚至可以包含列表 (所谓嵌套)。
- 列表是写在方括号 `[]` 之间、用逗号分隔开的元素列表。
- 列表索引值以 0 为开始值, -1 为从末尾的开始位置。
- 列表可以使用 `+` 操作符进行拼接, 使用 `*` 表示重复。

```
>>> list = ['abcd', 786 , 2.23, 'runoob', 70.2]
>>> print(list[1:3])
[786, 2.23]
>>> tinylist = [123, 'runoob']
>>> print(list + tinylist)
['abcd', 786, 2.23, 'runoob', 70.2, 123, 'runoob']
```

3.2.1 列表的定义与访问

- 列表的格式

变量A的类型为列表

```
namesList = ['xiaowang', 'xiaoZhang', 'xiaoHua']
```

比C语言的数组强大的地方在于列表中的元素可以是不同类型的

```
testList = [1, 'a']
```

- 打印列表

demo:

```
namesList = ['xiaowang', 'xiaoZhang', 'xiaoHua']  
print(namesList[0])  
print(namesList[1])  
print(namesList[2])
```

- 列表的循环遍历

为了更有效率的输出列表的每个数据，可以使用循环来完成

demo:

```
namesList = ['xiaowang', 'xiaoZhang', 'xiaoHua']  
for name in namesList:  
    print(name)
```

3.2.3 常用操作

-----操作名称----- -----	操作方法	举例
访问列表中的元素	通过下标直接访问	<code>print(list1[0])</code>
列表的切片	使用 <code>[:]</code>	<code>list1[2:5:2]</code>
遍历列表	通过for循环	<code>for i in list1: print(i)</code>
【增】新增数据到列表尾部	使用append	<code>list1.append(5)</code>
【增】列表的追加	使用extend方法	<code>list1.extend(list2)</code>
【增】列表数据插入	insert方法	<code>list1.insert(1, 3)</code>
【删】列表的删除	del：我们通过索引删除指定位置的元素。 remove：移除列表中指定值的第一个匹配值。如果没找到的话，会抛异常。	<code>del list1[0]</code> <code>list1.remove(1)</code> 注意两种方法的区别
【删】弹出列表尾部元素	使用pop	<code>list1.pop()</code>
【改】更新列表中的数据	通过下标原地修改	<code>list1[0] = 8</code>
【查】列表成员关系	in 、 not in	<code>2 in list1</code>
列表的加法操作	+	<code>list3 = list1 + list2</code>
【排】列表的排序	sort方法	<code>list1.sort()</code>
【排】列表的反转	reverse	<code>list1.reverse()</code>

操作名称	操作方法	举例
获取列表长度	len ()	
获取列表元素最大值	max ()	
获取列表元素最小值	min ()	
其它类型对象转换成列表	list ()	

3.2.4 列表的嵌套

- 列表嵌套

类似while循环的嵌套，列表也是支持嵌套的

一个列表中的元素又是一个列表，那么这就是列表的嵌套

```
schoolNames = [['北京大学', '清华大学'],  
                ['南开大学', '天津大学', '天津师范大学'],  
                ['山东大学', '中国海洋大学']]
```

- 应用

一个学校，有3个办公室，现在有8位老师等待工位的分配，请编写程序，完成随机的分配

```
#encoding=utf-8  
  
import random  
  
# 定义一个列表用来保存3个办公室  
offices = [[], [], []]  
  
# 定义一个列表用来存储8位老师的名字  
names = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']  
  
i = 0  
for name in names:  
    index = random.randint(0,2)  
    offices[index].append(name)  
  
i = 1  
for tempNames in offices:  
    print('办公室%d的人数为:%d'%(i, len(tempNames)))  
    i+=1  
    for name in tempNames:  
        print("%s"%name, end=' ')  
    print("\n")  
    print("-"*20)
```

作业:

现有商品列表如下:

1. products = [["iphone",6888],["MacPro",14800],["小米6",2499],["Coffee",31],["Book",60],
["Nike",699]], 需打印出以下格式:

```
----- 商品列表 -----  
0  iphone    6888  
1  MacPro    14800  
2  小米6     2499  
3  Coffee    31  
4  Book      60  
5  Nike      699
```

2. 根据上面的products列表写一个循环, 不断询问用户想买什么, 用户选择一个商品编号, 就把对应的商品添加到购物车里, 最终用户输入q退出时, 打印购买的商品列表。

3.3 元组

🧩 Python的核心数据类型

◆ Tuple (元组)

- tuple与list类似, 不同之处在于tuple的元素不能修改。tuple写在小括号里, 元素之间用逗号隔开。
- 元组的元素不可变, 但可以包含可变对象, 如list。

⚠ **注意:** 定义一个只有1个元素的tuple, 必须加逗号。

```
>>> t = ('abcd', 786, 2.23, 'runoob', 70.2)  
>>> t1 = (1, )  
>>> t2 = ('a', 'b', ['A', 'B'])  
>>> t[2][0] = 'X'  
>>> t  
(('a', 'b', ['X', 'B'])
```

3.3.1 元组的定义与访问

- 创建空元组

```
tup1 = ()
```

- 元组的定义

```
tup1 = (50)          # 不加逗号，类型为整型
print(type(tup1))    # 输出<class 'int'>
```

```
tup1 = (50,)         # 加逗号，类型为元组
print(type(tup1))    # 输出<class 'tuple'>
```

- 元组的访问

```
tup1 = ('Google', 'baidu', 2000, 2020)
tup2 = (1, 2, 3, 4, 5, 6, 7)

print ("tup1[0]: ", tup1[0])
print ("tup2[1:5]: ", tup2[1:5])
```

- 元组中的元素值是不允许修改的，但我们可以对元组进行连接组合，如下实例:

```
tup1 = (12, 34.56)
tup2 = ('abc', 'xyz')

# 以下修改元组元素操作是非法的。
# tup1[0] = 100

# 创建一个新的元组
tup3 = tup1 + tup2
print (tup3)
```

- 删除元组后，再次访问会报错

```
tup = ('Google', 'baidu', 2000, 2020)

print (tup)
del tup
print ("删除后的元组 tup : ")
print (tup)
```

3.3.2 常用操作:

操作名称	操作方法	举例
访问元组中的元素	通过下标直接访问	print(tuple1[0])
遍历元组	通过for循环	for i in tuple1: print(i)
元组的切片	使用[:]	tuple1[2:5:2]

操作名称	操作方法	举例
元组的加法操作		tuple3 = tuple1 + tuple2

操作名称	操作方法	举例
元组成员关系	in	2 in list1
得到重复元素数量	count	tuple1.count(1)

操作名称	操作方法	举例
获取元组长度	len ()	
获取元组元素最大值	max ()	
获取元组元素最小值	min ()	
其它类型对象转换成元组	tuple ()	

3.4 字典

Python的核心数据类型

◆ dict (字典)

- 字典是无序的对象集合，使用键-值 (key-value) 存储，具有极快的查找速度。
- 键(key)必须使用不可变类型。
- 同一个字典中，键(key)必须是唯一的。

```
>>> d = {'Michael': 95, 'Bob': 75, 'Tracy': 85}
>>> d['Michael']
95
```

3.4.1 字典的定义

变量info为字典类型：

```
info = {'name': '班长', 'id': 100, 'sex': 'f', 'address': '地球亚洲中国北京'}
```

说明：

- 字典和列表一样，也能够存储多个数据
- 列表中找某个元素时，是根据下标进行的
- 字典中找某个元素时，是根据'名字'（就是冒号:前面的那个值，例如上面代码中的'name'、'id'、'sex'）
- 字典的每个元素由2部分组成，键:值。例如 'name': '班长', 'name'为键，'班长'为值

3.4.2 根据键访问值

```
info = {'name': '吴彦祖', 'age': 18}

print(info['age']) # 获取年龄

# print(info['sex']) # 获取不存在的key，会发生异常

print(info.get('sex')) # 获取不存在的key，获取到空的内容，不会出现异常
```

若访问不存在的键，则会报错：

```
info['sex']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'sex'
```

在我们不确定字典中是否存在某个键而又想获取其值时，可以使用get方法，还可以设置默认值：

```
>>> age = info.get('age')
>>> age # 'age'键不存在，所以age为None
>>> type(age)
<type 'NoneType'>
>>> age = info.get('age', 18) # 若info中不存在'age'这个键，就返回默认值18
>>> age
18
```

3.4.3 常用操作：

-----操作名称 -----	操作方法	举例
访问字典中的元素 (1)	通过key访问, key不存在会抛出异常	<code>print(dict1["小明"])</code>
访问字典中的元素 (2)	通过get方法, 不存在返回None, 不抛出异常	<code>print(dict1.get("小明"))</code>
遍历字典 (1)	通过for循环, 只能获取key	<code>for key in dict1: print(key, dict1[key])</code>
遍历字典 (2)	配合items方法, 获取key和val	<code>for key, val in dict1.items(): print(key, val)</code>
直接获取所有key和value	使用keys和values方法	<code>print(dict1.values()) print(dict1.keys())</code>
修改val	直接通过key修改	<code>dict1["小明"] = 2003</code>
新增键值对	直接新增	<code>dict1["小李"] = 2005</code>

操作名称	操作方法	举例
字典元素的删除	通过key删除	<code>del dict1["小李"]</code>
字典元素的弹出	通过pop方法	<code>dict1.pop("小李")</code>
判断key是否存在	in	<code>"key" in dict1</code>
合并字典	update	<code>dict1.update(dict2)</code>
把两个列表转为字典	dict+zip方法	<code>dict(zip(list1, list2))</code>
把一个嵌套列表转为字典	dict方法	<code>dict2 = dict([['key1', 'value1']])</code>
清除字典内的元素	clear方法	<code>dict1.clear()</code>

操作名称	操作方法	举例
获取字典长度	len ()	
获取最大的key	max ()	
获取最小的key	min ()	
其它类型对象转换成字典	dict ()	dict([(1, 2), (2, 3)])

3.5 集合

Python的核心数据类型

◆ set (集合)

- set和dict类似，也是一组key的集合，但不存储value。由于key不能重复，所以，在set中，没有重复的key。
- set是无序的，重复元素在set中自动被过滤。

```
>>> s = set([1, 2, 3])
>>> s
{1, 2, 3}
>>> s = set([1, 1, 2, 2, 3, 3])
>>> s
{1, 2, 3}
```

set可以看成数学意义上的无序和无重复元素的集合，因此，两个set可以做数学意义上的交集（&）、并集（|）、差集（-）等操作。

常用操作：

操作名称	操作方法	举例
遍历集合	通过for循环	for i in set1: print(i)
更新集合	update方法	set1.update(set2)
向集合添加新元素	add方法	set1.add(5)
移除集合中的元素	remove方法	set1.remove(5)
弹出元素	pop方法	val = set1.pop()
清除元素	clear方法	set1.clear()
删除集合	del	del set1

操作名称	操作方法	举例
获取集合长度	len ()	
获取最大的元素	max ()	
获取最小的元素	min ()	
其它类型对象转换成集合	set ()	

3.6小结

	是否有序	是否可变类型
列表[]	有序	可变类型
元组()	有序	不可变类型
字典{}	无序	key不可变 val可变
集合{}	无序	可变类型（不重复）

4.函数

4.1 函数的概念

如果在开发程序时，需要某块代码多次，但是为了提高编写的效率以及代码的重用，所以把具有独立功能的代码块组织为一个小模块，这就是函数。

4.2 函数定义和调用

4.2.1 定义函数

定义函数的格式如下：

```
def 函数名():  
    代码
```

demo:

```
# 定义一个函数，能够完成打印信息的功能
def printInfo():
    print '-----'
    print '          人生苦短，我用Python'
    print '-----'
```

4.2.2 调用函数

定义了函数之后，就相当于有了一个具有某些功能的代码，想要让这些代码能够执行，需要调用它
调用函数很简单的，通过 **函数名()** 即可完成调用

demo:

```
# 定义完函数后，函数是不会自动执行的，需要调用它才可以
printInfo()
```

4.3 函数参数

4.3.1 定义带有参数的函数

示例如下:

```
def add2num(a, b):
    c = a+b
    print c
```

4.3.2 调用带有参数的函数

以调用上面的add2num(a, b)函数为例:

```
def add2num(a, b):
    c = a+b
    print c

add2num(11, 22) #调用带有参数的函数时，需要在小括号中，传递数据
```

调用带有参数函数的运行过程：



```
#定义接收2个参数的函数
def add2num( a , b ):
    c = a+b
    print c
```

```
#调用带有参数的函数
add2num(110, 22)
```

终端

4.4 函数返回值

4.4.1 带有返回值的函数

想要在函数中把结果返回给调用者，需要在函数中使用return

如下示例：

```
def add2num(a, b):
    c = a+b
    return c
```

或者

```
def add2num(a, b):
    return a+b
```

4.4.2 保存函数的返回值

在本小节刚开始的时候，说过的“买烟”的例子中，最后儿子给你烟时，你一定是从儿子手中接过来 对么，程序也是如此，如果一个函数返回了一个数据，那么想要用这个数据，那么就需要保存

保存函数的返回值示例如下：

```
#定义函数
def add2num(a, b):
    return a+b

#调用函数，顺便保存函数的返回值
result = add2num(100,98)

#因为result已经保存了add2num的返回值，所以接下来就可以使用了
print result
```

4.4.3在python中我们可不可以返回多个值？（了解）

```
>>> def divid(a, b):
...     shang = a//b
...     yushu = a%b
...     return shang, yushu
...
>>> sh, yu = divid(5, 2)
>>> sh
5
>>> yu
1
```

本质是利用了元组

课堂练习：

- 1.写一个打印一条横线的函数。（提示：横线是若干个“-”组成）
- 2.写一个函数，可以通过输入的参数，打印出自定义行数的横线。（提示：调用上面的函数）
- 3.写一个函数求三个数的和
- 4.写一个函数求三个数的平均值（提示：调用上面的函数）

【建议每题5分钟以内】

参考代码1-2:

```
# 打印一条横线
def printOneLine():
    print("-"*30)

# 打印多条横线
def printNumLine(num):
    i=0

    # 因为printOneLine函数已经完成了打印横线的功能,
    # 只需要多次调用此函数即可
    while i<num:
        printOneLine()
        i+=1

printNumLine(3)
```

参考代码3-4:

```
# 求3个数的和
def sum3Number(a,b,c):
    return a+b+c # return 的后面可以是数值,也可是一个表达式

# 完成对3个数求平均值
def average3Number(a,b,c):

    # 因为sum3Number函数已经完成了3个数的求和,所以只需调用即可
    # 即把接收到的3个数,当做实参传递即可
    sumResult = sum3Number(a,b,c)
    aveResult = sumResult/3.0
    return aveResult

# 调用函数,完成对3个数求平均值
result = average3Number(11,2,55)
print("average is %d"%result)
```

4.5 局部变量和全局变量

4.5.1 局部变量

什么是局部变量

如下图所示:

```
test.py - Sublime Text
1 #coding=utf-8
2
3 def test1():
4     a = 300
5     print('----test1--修改前--a=%d'%a)
6     a = 200
7     print('----test1--修改后--a=%d'%a)
8
9 def test2():
10    a = 400
11    print('----test3----a=%d'%a)
12
13
14 # 调用函数
15 test1()
16 test2()
17
```

```
python@ubuntu: ~/Desktop
python@ubuntu:~/Desktop$ python3 test.py
----test1--修改前--a=300
----test1--修改后--a=200
----test3----a=400
python@ubuntu:~/Desktop$
```

小总结

- 局部变量，就是在函数内部定义的变量
- 不同的函数，可以定义相同的名字的局部变量，但是各用个的不会产生影响
- 局部变量的作用，为了临时保存数据需要在函数中定义变量来进行存储，这就是它的作用

4.5.2 全局变量

什么是全局变量

如果一个变量，既能在一个函数中使用，也能在其他的函数中使用，这样的变量就是 **全局变量**

demo如下:

```
# 定义全局变量
a = 100

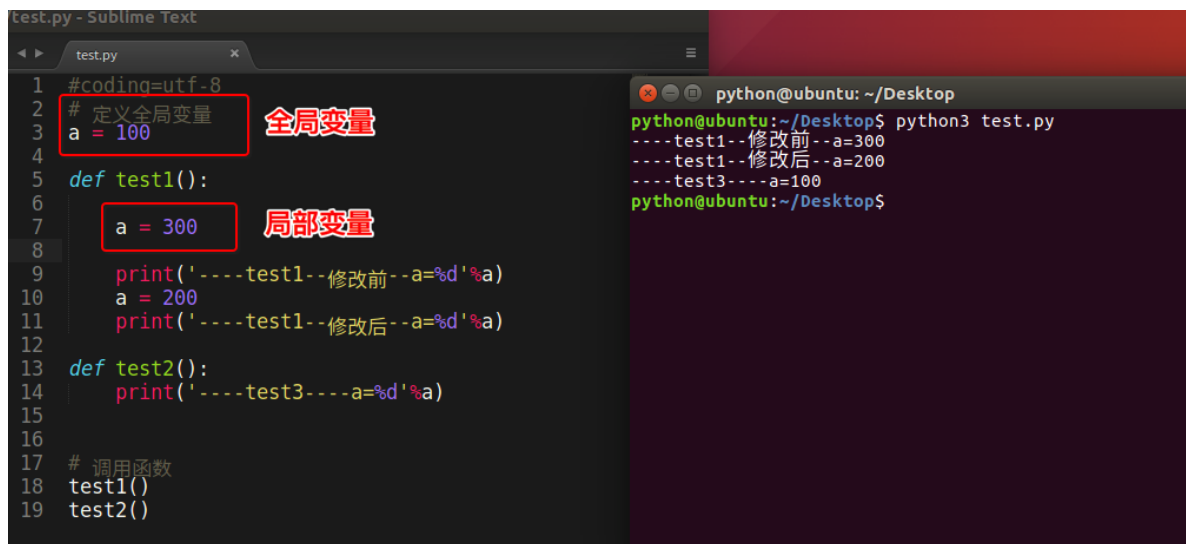
def test1():
    print(a)

def test2():
    print(a)

# 调用函数
test1()
test2()
```

全局变量和局部变量名字相同问题

看如下代码:



```
test.py - Sublime Text
1 #coding=utf-8
2 # 定义全局变量
3 a = 100
4
5 def test1():
6     a = 300
7     print('----test1--修改前--a=%d'%a)
8     a = 200
9     print('----test1--修改后--a=%d'%a)
10
11 def test2():
12     print('----test3----a=%d'%a)
13
14 # 调用函数
15 test1()
16 test2()
```

```
python@ubuntu: ~/Desktop
python@ubuntu:~/Desktop$ python3 test.py
----test1--修改前--a=300
----test1--修改后--a=200
----test3----a=100
python@ubuntu:~/Desktop$
```

修改全局变量

既然全局变量，就是能够在所有的函数中进行使用，那么可否进行修改呢？

代码如下：



```
test.py - Sublime Text
1 #coding=utf-8
2 # 定义全局变量
3 a = 100
4
5 def test1():
6     global a
7     print('----test1--修改前--a=%d'%a)
8     a = 200
9     print('----test1--修改后--a=%d'%a)
10
11 def test2():
12     print('----test3----a=%d'%a)
13
14 # 调用函数
15 test1()
16 test2()
```

```
python@ubuntu: ~/Desktop
python@ubuntu:~/Desktop$ python3 test.py
----test1--修改前--a=100
----test1--修改后--a=200
----test3----a=200
python@ubuntu:~/Desktop$
```

程序ok,
没有出错

总结:

- 在函数外边定义的变量叫做 全局变量
- 全局变量能够在所有的函数中进行访问
- 如果在函数中修改全局变量，那么就需要使用 `global` 进行声明，否则出错
- 如果全局变量的名字和局部变量的名字相同，那么使用的是局部变量的，小技巧 强龙不压地头蛇

4.6 函数使用注意事项

1. 自定义函数

<1>无参数、无返回值

```
def 函数名():  
    语句
```

<2>无参数、有返回值

```
def 函数名():  
    语句  
    return 需要返回的数值
```

注意:

- 一个函数到底有没有返回值，就看有没有return，因为只有return才可以返回数据
- 在开发中往往根据需求来设计函数需不需要返回值
- 函数中，可以有多个return语句，但是只要执行到一个return语句，那么就意味着这个函数的调用完成

<3>有参数、无返回值

```
def 函数名(形参列表):  
    语句
```

注意:

- 在调用函数时，如果需要把一些数据一起传递过去，被调用函数就需要用参数来接收
- 参数列表中变量的个数根据实际传递的数据的多少来确定

<4>有参数、有返回值

```
def 函数名(形参列表):  
    语句  
    return 需要返回的数值
```

<5>函数名不能重复

2. 调用函数

<1>调用的方式为:

```
函数名([实参列表])
```

<2>调用时，到底写不写 实参

- 如果调用的函数 在定义时有形参，那么在调用的时候就应该传递参数

<3>调用时，实参的个数和先后顺序应该和定义函数中要求的一致

<4>如果调用的函数有返回值，那么就可以用一个变量来进行保存这个值

3. 作用域

<1>在一个函数中定义的变量，只能在本函数中用(局部变量)

<2>在函数外定义的变量，可以在所有的函数中使用(全局变量)

5.文件操作

文件，就是把一些数据存放起来，可以让程序下一次执行的时候直接使用，而不必重新制作一份，省时省力。

5.1 文件打开与关闭

5.1.1 打开文件

在python，使用open函数，可以打开一个已经存在的文件，或者创建一个新文件

open(文件名，访问模式)

示例如下：

```
f = open('test.txt', 'w')
```

说明：

访问模式	说明
r	以只读方式打开文件。文件的指针将会放在文件的开头。这是默认模式。
w	打开一个文件只用于写入。如果该文件已存在则将其覆盖。如果该文件不存在，创建新文件。
a	打开一个文件用于追加。如果该文件已存在，文件指针将会放在文件的结尾。也就是说，新的内容将会被写入到已有内容之后。如果该文件不存在，创建新文件进行写入。
rb	以二进制格式打开一个文件用于只读。文件指针将会放在文件的开头。这是默认模式。
wb	以二进制格式打开一个文件只用于写入。如果该文件已存在则将其覆盖。如果该文件不存在，创建新文件。
ab	以二进制格式打开一个文件用于追加。如果该文件已存在，文件指针将会放在文件的结尾。也就是说，新的内容将会被写入到已有内容之后。如果该文件不存在，创建新文件进行写入。
r+	打开一个文件用于读写。文件指针将会放在文件的开头。
w+	打开一个文件用于读写。如果该文件已存在则将其覆盖。如果该文件不存在，创建新文件。
a+	打开一个文件用于读写。如果该文件已存在，文件指针将会放在文件的结尾。文件打开时会是追加模式。如果该文件不存在，创建新文件用于读写。
rb+	以二进制格式打开一个文件用于读写。文件指针将会放在文件的开头。
wb+	以二进制格式打开一个文件用于读写。如果该文件已存在则将其覆盖。如果该文件不存在，创建新文件。
ab+	以二进制格式打开一个文件用于追加。如果该文件已存在，文件指针将会放在文件的结尾。如果该文件不存在，创建新文件用于读写。

5.1.2 关闭文件

`close()`

示例如下：

```
# 新建一个文件，文件名为:test.txt
f = open('test.txt', 'w')

# 关闭这个文件
f.close()
```

5.2 文件读写

5.2.1 写数据(write)

使用write()可以完成向文件写入数据

demo:

```
f = open('test.txt', 'w')
f.write('hello world, i am here!')
f.close()
```

注意:

- 如果文件不存在那么创建, 如果存在那么就先清空, 然后写入数据

5.2.2 读数据(read)

使用read(num)可以从文件中读取数据, num表示要从文件中读取的数据的长度(单位是字节), 如果没有传入num, 那么就表示读取文件中所有的数据

demo:

```
f = open('test.txt', 'r')

content = f.read(5)

print(content)

print("-"*30)

content = f.read()

print(content)

f.close()
```

注意:

- 如果open是打开一个文件, 那么可以不用写打开的模式, 即只写 `open('test.txt')`
- 如果使用读了多次, 那么后面读取的数据是从上次读完后的位置开始的

5.2.3 读数据 (readlines)

就像read没有参数时一样, readlines可以按照行的方式把整个文件中的内容进行一次性读取, 并且返回的是一个列表, 其中每一行的数据为一个元素

```
#coding=utf-8

f = open('test.txt', 'r')

content = f.readlines()

print(type(content))

i=1
for temp in content:
    print("%d:%s"%(i, temp))
    i+=1
```

```
f.close()
```

5.2.4 读数据 (readline)

```
#coding=utf-8

f = open('test.txt', 'r')

content = f.readline()
print("1:%s"%content)

content = f.readline()
print("2:%s"%content)

f.close()
```

5.3 文件的相关操作

有些时候，需要对文件进行重命名、删除等一些操作，python的os模块中都有这么功能

5.3.1 文件重命名

os模块中的rename()可以完成对文件的重命名操作

rename(需要修改的文件名, 新的文件名)

```
import os

os.rename("毕业论文.txt", "毕业论文-最终版.txt")
```

5.3.2 删除文件

os模块中的remove()可以完成对文件的删除操作

remove(待删除的文件名)

```
import os

os.remove("毕业论文.txt")
```

5.3.3 创建文件夹

```
import os

os.mkdir("张三")
```

5.3.4 获取当前目录


```
import os

os.getcwd()
```

5.3.5 改变默认目录

```
import os

os.chdir("../")
```

5.3.6 获取目录列表

```
import os

os.listdir("../")
```

5.3.7 删除文件夹

```
import os

os.rmdir("张三")
```

6. 错误与异常

6.1 异常简介

看如下示例:

```
print '-----test--1---'
open('123.txt','r')
print '-----test--2---'
```

运行结果:

```
code@ubuntu:~/python-test$ python test.py
-----test--1---
Traceback (most recent call last):
  File "test.py", line 2, in <module>
    open('123.txt','r')
IOError: [Errno 2] No such file or directory: '123.txt'
```

说明:

打开一个不存在的文件123.txt, 当找不到123.txt 文件时, 就会抛出给我们一个IOError类型的错误, No such file or directory: 123.txt (没有123.txt这样的文件或目录)

异常:

当Python检测到一个错误时，解释器就无法继续执行了，反而出现了一些错误的提示，这就是所谓的"异常"

6.2 捕获异常 try...except...

看如下示例:

```
try:
    print('-----test--1---')
    open('123.txt','r')
    print('-----test--2---')
except IOError:
    pass
```

说明:

- 此程序看不到任何错误，因为用except 捕获到了IOError异常，并添加了处理的方法
- pass 表示实现了相应的实现，但什么也不做；如果把pass改为print语句，那么就会输出其他信息

小总结:

The diagram shows the code from the previous example with two red boxes and arrows explaining its parts:

- A red box around the `try:` block (containing `print`, `open`, and `print` statements) is pointed to by an arrow from a red box containing the text: **可能产生异常的代码，放在try中** (Code that may cause an exception, put in try).
- A red box around the `except IOError:` block (containing `pass`) is pointed to by an arrow from a red box containing the text: **如果产生错误时，处理的方法** (Method of handling when an error occurs).

- 把可能出现问题的代码，放在try中
- 把处理异常的代码，放在except中

6.3 except捕获多个异常

看如下示例:

```
try:
    print num
except IOError:
    print('产生错误了')
```

想一想:

上例程序，已经使用except来捕获异常了，为什么还会看到错误的信息提示？

答:

except捕获的错误类型是IOError，而此时程序产生的异常为 NameError，所以except没有生效

修改后的代码为:

```
try:
    print num
except NameError:
    print('产生错误了')
```

实际开发中, 捕获多个异常的方式, 如下:

```
#coding=utf-8
try:
    print('-----test--1---')
    open('123.txt','r') # 如果123.txt文件不存在, 那么会产生 IOError 异常
    print('-----test--2---')
    print(num) # 如果num变量没有定义, 那么会产生 NameError 异常

except (IOError,NameError):
    #如果想通过一次except捕获到多个异常可以用一个元组的方式
```

注意: **

- 当捕获多个异常时, 可以把要捕获的异常的名字, 放到except后, 并使用元组的方式仅进行存储

6.4 获取异常的信息描述

```
In [8]: try:
...:     print(a)
...: except NameError as result:
...:     print(result)
```

name 'a' is not defined

存储异常的
基本信息

要捕获的异常

```
In [13]: try:
...:     open("a.txt")
...: except (NameError,IOError) as result:
...:     print("哈哈, 捕获到了异常")
...:     print("异常的基本信息是:",result)
```

哈哈, 捕获到了异常
异常的基本信息是: [Errno 2] No such file or directory: 'a.txt'

捕获多个异常, 并且存储异常的基本信息

6.5 捕获所有异常

```
In [14]: try:
...:     open("a.txt")
...: except:
...:     print("产生了一个异常")
...:
产生了一个异常
```

没有存储异常
的基本信息

```
In [15]: try:
...:     open("a.txt")
...: except Exception as result:
...:     print("捕获到了异常")
...:     print(result)
...:
捕获到了异常
[Errno 2] No such file or directory: 'a.txt'
```

捕获所有异常，
并且存储异常的基本信息

6.6 try...finally...

try...finally...语句用来表达这样的情况：

在程序中，如果一个段代码必须要执行，即无论异常是否产生都要执行，那么此时就需要使用 finally。比如文件关闭，释放锁，把数据库连接返还给连接池等

demo:

```
import time
try:
    f = open('test.txt')
    try:
        while True:
            content = f.readline()
            if len(content) == 0:
                break
            time.sleep(2)
            print(content)
    except:
        #如果在读取文件的过程中，产生了异常，那么就会捕获到
        #比如 按下了 ctrl+c
        pass
    finally:
        f.close()
        print('关闭文件')
except:
    print("没有这个文件")
```

说明:

test.txt文件中每一行数据打印，但是 I 有意在每打印一行之前用 time.sleep 方法暂停 2 秒钟。这样做的原因是让程序运行得慢一些。在程序运行的时候，按 Ctrl+c 中断（取消）程序。

我们可以观察到 KeyboardInterrupt 异常被触发，程序退出。但是在程序退出之前，finally 从句仍然被执行，把文件关闭。

作业：

1. 应用文件操作的相关知识，通过Python新建一个文件gushi.txt，选择一首古诗写入文件中
2. 另外写一个函数，读取指定文件gushi.txt，将内容复制到copy.txt中，并在控制台输出“复制完毕”。
3. 提示：分别定义两个函数，完成读文件和写文件的操作
尽可能完善代码，添加异常处理。

IT私塾课件